

AI 助手项目规则

Claude Code（以及类似的 AI Agent）会自动读取本文件。如果你是正在此仓库中 工作的 AI，请直接遵守以下规则，无需另行请求许可；这些规则体现了维护者 已经明确作出的决定。

硬性规则：本地优先

每一项改动都必须严格遵循以下路径：

```
本地工作副本（D:\Work\LewisDocs\）
  → npm run build    （在本地验证）
  → git add + git commit
  → git push origin main
  → GitHub Actions CI
  → Cloudflare Pages 部署
  → https://lewisdocs.pages.dev/
```

绝不允许：

- ✗ 在 GitHub Web UI 中编辑（Edit this file 按钮）——这会绕过本地验证
- ✗ 修改 Cloudflare Pages 控制台（Settings > Headers / Variables / Functions）——这会与 Git 中的配置失去同步
- ✗ 直接修改 docs/.vitepress/dist/——下次构建时会被覆盖
- ✗ 修改 docs/index.md / docs/theory.md / docs/comparison.md / docs/platforms/*.md——这些文件由 npm run prepare-content 根据 source/ 派生生成，手动修改会被清除
- ✗ 对于可以通过仓库文件完成的改动，建议用户在任何 Web UI 中手动点击操作

在 D:\Work\LewisDocs\ 中工作时，始终直接通过 git push 推送到 origin main（即 https://github.com/Lewis0x/LewisDocs.git）。不要使用 /tmp 克隆，也不要手动同步。

注释等级（何时为正文中的术语添加注释）

网站读者包括**管理者**和**决策者**，而不只是同领域的 CAD 架构师。应解释 CTO 可能需要说明的术语，但不要过度注释。判断流程如下：

它是品牌或产品名（Cloudflare Pages、Mermaid、VitePress）吗？

它是 CAD 专用术语（B-Rep、Parasolid、FeatureScript、TNP）吗？

或 [链接]

它是通用缩写（API、SDK、IDE、CI）吗？

的缩写表，

不具备软件背景的决策者是否需要解释才能理解？

句中中文">

→ T4: 跳过

→ T1: 加入术语表，并添加<Term def="...">

→ T3: 只加入术语表

不在正文中标注

→ T2: 在正文中添加<Term def="一

（出现 3 次以上

时升级到术语表）

其他情况

→ 不标注

密度规则：

- 每章只在首次出现时标注（不是每页或每段都标注）
- 跳过代码块、标题、表头、^[百科 N]、[回链：...] 和 Mermaid 节点
- 每段最多添加 2 个注释

定义的写作风格：

- 1~2 句话，30~80 个中文字符，采用“是什么 + 关键特征”的结构
- 不引用其他章节（避免在工具提示中嵌套链接）
- 不写只有英文的定义

运行 `python scripts/find_jargon.py`，查找尚未添加注释的术语候选项。完整方法见 `project-docs/05-annotation-methodology.md`。

事实来源映射

类型	路径	何时修改
源内容	<code>source/*.md</code>	更新 13 份研究文档时（9 份平台深度分析 3.1~3.9 + 文档 0/1/2/4 + 术语表）
网站配置	<code>docs/.vitepress/config.ts</code>	修改导航、侧边栏、搜索、Mermaid 或 head 元数据时
主题	<code>docs/.vitepress/theme/</code>	修改自定义 Vue 组件（ <code>components/</code> ）、 <code>custom.css</code> 或 <code>index.ts</code> 时
公共资源	<code>docs/public/</code>	修改 <code>robots.txt</code> 、 <code>ai.txt</code> 、 <code>_headers</code> 、 <code>_honeypot/</code> 时
处理流水线	<code>scripts/*.py</code>	修改导入、链接重写、引用链接、水印或审计逻辑时
CI	<code>.github/workflows/*.yaml</code>	修改构建和部署流水线时（CF Pages 已启用，GH Pages 已禁用）
GitLab 后备方案	<code>.gitlab-ci.yml</code>	修改可选的 GitLab Pages 镜像时
项目元信息	<code>project-docs/</code> 、 <code>README.md</code> 、 <code>LICENSE</code> 、 <code>CLAUDE.md</code> （本文件）	修改流程、需求、设计、开发、使用、注释或引用审计说明时
派生文件（禁止修	<code>docs/index.md</code> 、 <code>docs/theory.md</code> 、 <code>docs/comparison.md</code> 、 <code>docs/glossary.md</code> 、	—— 由 <code>npm run prepare-content</code> 生成

类型	路径	何时修改
改)	docs/platforms/*.md	
派生文件 (禁止修改)	docs/.vitepress/dist/ docs/.vitepress/cache/	—— 由 <code>npm run build</code> 生成

构建流程（各步骤的作用）

<code>npm run prepare-content</code> <code>scripts/import_docs.py</code> <code>source/*.md</code> → <code>docs/*.md</code>（适合路由的文件名 + frontmatter） <code>scripts/rewrite_links.py</code>（slugify 必须与 VitePress 一致） <code>scripts/link_citations.py</code> <code>[百科 N] / [新闻 N] / [官方 N] / [第三方 N] / [书籍 N]</code> → <code>[百科 N]</code>	<code>[回链: 3.4 §三 ...]</code> → 实际 Markdown 链接 <code>[百科 N] / [新闻 N] / [官方 N] / [第三方 N] / [书籍 N]</code> → <code>[百科 N]</code> （读取每篇文档的 <code>`## 参考来源`</code> 章节，构建 <code>(kind,num)→URL</code> 映射）
<code>npm run build</code> <code>vitepress build docs</code> <code>scripts/watermark.py</code>	静态网站 → <code>docs/.vitepress/dist/</code> (1) 移除 VitePress generator meta 标签 (2) 为每个页面注入零宽字符水印， 生成 <code>docs/.vitepress/dist/_watermark-</code>
<code>manifest.json</code>	CI 将清单作为保留 90 天的构件上传，然后在部署 CF
<code>Pages</code>	之前执行 <code>`rm -f`</code> ；清单属于私密文件。

`npm run build:no-watermark` 仅用于本地调试；绝不能提交或部署没有水印的版本。

辅助脚本（不在构建流程中）

脚本	用途
<code>scripts/find_jargon.py</code>	扫描 <code>source/</code> 中未添加注释的术语 → <code>jargon-report.md</code> （已被 Git 忽略）。由人工判断 T1~T4。
<code>scripts/audit_citations.py</code>	从源文档中提取每个引用 <code>[kind N]</code> → <code>audit_report.csv / .md</code> （已被 Git 忽略）。 <code>--check-urls</code> 会执行 HTTP HEAD 探测。处理过程记录在 <code>project-docs/06-citation-audit.md</code> 。
<code>scripts/fix_term_quotes.py</code>	修复 <code><Term def="..."></code> 内部的 ASCII "（否则 Vue HTML 解析器会出错，将其转换为 U+201C/U+201D）。
<code>scripts/scan_corpus.py</code>	水印溯源工具。输入可疑语料和清单，返回泄漏来源的（ <code>page_path</code> ， <code>build_id</code> ）。

30 秒了解架构

- 使用 VitePress 1.5+ 构建静态网站（通过 `vitepress-plugin-mermaid` 绘制图表），源内容为 13 份 Markdown 研究文档（约 20.5 万字；9 份平台深度分析 + 理论 + 对比 + UI 研究 + 术语表 + 索引）
- 搜索使用本地索引；中文在浏览器中通过 `Intl.Segmenter` 分词，在构建阶段的 Node 环境中使用正则表达式作为后备方案
- 托管在 **Cloudflare Pages**（项目名：`lewisdocs`，地址：<https://lewisdocs.pages.dev/>）
- CI 使用 **GitHub Actions**（[.github/workflows/cloudflare-pages.yml](https://github.com/lewisdocs/workflows/cloudflare-pages.yml)）；并发组为 `cloudflare-pages`
- 反爬虫包含 5 层措施（`robots.txt` + 包括 AI 机器人的 `noindex meta` + 边缘 `_headers` + `_honeypot/` + 零宽字符水印 + 带有 AI 使用限制的 CC BY-NC-ND 4.0 LICENSE）——见 [project-docs/02-design.md](#) 中的 ADR-009
- 旧的 GitHub Pages 工作流已禁用（`pages.yml` 中设置了 `if: false`）；`.gitlab-ci.yml` 是并行的后备方案

不确定时

1. 先阅读 [project-docs/03-development.md](#)——其中包含完整工作流、常见问题和调试技巧
2. 阅读 [project-docs/02-design.md](#)，了解架构决策依据（8 项 ADR）
3. 添加任何 `<Term>` 或术语表条目之前，先阅读 [project-docs/05-annotation-methodology.md](#)
4. 修改源文档中的 `## 参考来源` 章节之前，先阅读 [project-docs/06-citation-audit.md](#)
5. 如果用户的要求与这些规则冲突，应在执行前明确指出冲突，不要在不说明的情况下违反规则

常见问题（均来自实际经验）

- **VitePress 搜索索引为空** → 检查 `markdown.anchor.permalink` 是否为 `false`（索引器需要标题内的 `<a href="#...">` 来划分章节）
- **首次加载页面时内容为空白** → 水印必须注入到 Vue 挂载点 `#app` 之外（当前由 `scripts/watermark.py` 注入到 `</body>` 之前）
- **找不到 Cloudflare 项目** → CF Pages 项目名必须使用小写字母（仓库名为 `LewisDocs`，但项目名使用 `lewisdocs`）
- **搜索“autocad”没有结果** → 参见上面的“VitePress 搜索索引为空”；问题与 `permalink` 有关，而不是分词器
- **.vitepress/ 的位置** → 必须放在 `docs/` 内，而不是仓库根目录（由于使用 `vitepress build docs`，VitePress 会把 `docs/` 视为项目根目录）
- **Slugify 结果不一致** → 如果锚点失效，请对比 `scripts/rewrite_links.py` 中的 `vitepress_slugify` 与 `node_modules/vitepress/dist/node/chunk-*.js`；二者必须产生完全一致的结果
- **Vue scoped CSS + `:global(body|html ...)` 会隐藏整个页面** → 绝不能在 `<style scoped>` 中编写 `:global(body.X) .scoped-Y { ... }`；Vue 3.5 会丢弃带作用域的子选择器，只生成 `body.X { display: none }`，导致所有带该 `body` 类的用户看到空白页面。****涉及 `body/html` 的规则应放在 `docs/.vitepress/theme/custom.css`（无作用域）中。完整复盘见 [project-docs/03-development.md §12.1](#)**
- **`<Term def="...">` 的 `def` 值包含内部 ASCII " 会破坏 Vue 解析器** → 构建会报 `Invalid end tag`。在 `def` 值内部使用全角引号 `“”`（U+201C/U+201D），或运行 `python scripts/fix_term_quotes.py` 自动转换
- **宽表格溢出到右侧大纲下方** → `flex` 项默认的 `min-width: auto`（即 `min-content`）无法为宽内容收缩。需要为 `flex` 项添加 `min-width: 0`，并用 `.table-scroll-wrapper` `div` 包裹表格（`markdown.config` 会自动完成）

- **Lightbox 显示空白 SVG 弹窗** → `v-html` 无法在 HTML 容器内渲染 SVG（命名空间不匹配）。改用 `cloneNode(true) + appendChild`
- **实时 UI 问题只会出现在 localStorage 中保留缓存状态的“老用户”端** → 部署 OutlineToggle / OutlineResizer / Lightbox / Term 的任何改动后，**必须使用带有旧 localStorage 状态的浏览器测试**，不能只测试新标签页。SSR HTML 正确 ≠ hydration 后页面渲染正确
- **任何祖先元素设置 overflow-x/y: auto 时，CSS position: sticky 都无法锚定到页面视口** → sticky 元素会锚定到最近的滚动容器，而不是视口。如果 `.table-scroll-wrapper` 设置了 `overflow-x: auto`，其中的 `thead { position: sticky; top: 64px }` 将固定在“包装器顶部 + 64”处，而不是“视口顶部 + 64”处，导致页面滚动时布局混乱。不要仅用 CSS 组合带 overflow 的包装器和 sticky thead；需要采用 JS 驱动的克隆 thead 方案
- **涉及布局、sticky 或滚动的 CSS PR 需要在实际浏览器中验证** → bundle-rule-check、HTML-structure-check 和 clean-build 都是必要条件，但并不充分。必须打开实际页面、滚动并目视检查（这个结论来自两天内的 PR #10/#13/#16）
- **_watermark-manifest.json 泄漏到部署产物中** → CI 必须在上传构件之后、执行 `pages deploy` 之前运行 `rm -f docs/.vitepress/dist/_watermark-manifest.json`。该清单是唯一能将水印映射到页面的文件；一旦泄漏，水印就失去作用
- **CF Pages 首次部署失败** → `Ensure project exists` 步骤使用 `pages project create lewisdocs - -production-branch=main` 和 `continue-on-error: true`；项目名必须为小写
- **CF Pages 部署失败并提示 Invalid commit message, must be valid UTF-8（代码 8000111）** → `git log -1 --format=%B` 正文中的 emoji 或 4 字节 UTF-8 字符会通过 HTTP 标头传递（RFC 7230 规定只能使用 ASCII）。文件可以正常上传，但创建部署记录时会被拒绝。**提交消息正文必须仅使用 ASCII**；emoji 只用于 PR 说明或 README。复盘见 [project-docs/03-development.md §12.8](#)

包含根因分析的完整复盘见 [project-docs/03-development.md §12](#)——其中每一条都曾造成实际停机。